RELATÓRIO TÉCNICO DE AUDITORIA E CORREÇÕES DA API

Plataforma Pulse SaaS / AgendeAi | Repositório: diogenesjcsilva-djcs/Saas-agendamentos
Data de Emissão: Agosto de 2026 | Status Geral: APROVADO & CORRIGIDO (100%)

1. Sumário Executivo

Foi realizada uma revisão aprofundada da arquitetura de endpoints da API (src/lib/api-routes.ts), avaliando vetores de risco em controle de acesso, exposição indevida de dados pessoais (LGPD), integridade transacional de banco de dados e tratamento defensivo de erros. Todas as falhas identificadas foram rigorosamente corrigidas e validadas através de testes automatizados e compilação estática TypeScript.

2. Matriz Detalhada de Vulnerabilidades e Ações Corretivas

[SEC-01] ALTA — **Exposição de Dados Pessoais de Clientes (LGPD / Broken Access Control)**

Rota(s): GET /api/bookings

Falha: O endpoint era público e permitia a qualquer pessoa listar agendamentos com nome, e-mail, telefone, endereço residencial e notas privadas dos clientes.

Correção: Rota protegida com authMiddleware. Prestadores só acessam agendamentos vinculados ao seu providerId e clientes só visualizam os seus próprios. Acessos indevidos bloqueados com HTTP 403.

[SEC-02] ALTA — **Ausência de Atomicidade Transacional no Banco de Dados**

Rota(s): POST /tenants, PUT /availability-rules, PUT /provider/settings

Falha: Queries de escrita executadas fora de transações. Se uma inserção falhasse (ex: e-mail duplicado de última hora ou erro no loop), deixava dados órfãos ou apagava regras de horários sem reinserir.

Correção: Implementado gerenciamento com pool.connect(), BEGIN, COMMIT e ROLLBACK em todas as operações compostas, garantindo consistência ACID no PostgreSQL.

[SEC-03] MÉDIA — **Vazamento de Detalhes Técnicos do Banco (Information Disclosure)**

Rota(s): POST /tenants, GET /categories, GET /providers, PUT /provider/settings

Falha: Tratamento de exceções retornava details: error.message no corpo HTTP 500, expondo nomes de tabelas, schemas e constraints internas da base PostgreSQL.

Correção: Respostas 500 sanitizadas para mensagens genéricas e amigáveis ao usuário. Logs detalhados de exceção mantidos estritamente no ambiente interno do servidor.

[SEC-04] ALTA — **Broken Object Level Authorization (BOLA / IDOR)**

Rota(s): PUT/DELETE /services/:id, DELETE /bookings/:id, DELETE /exceptions/:id

Falha: Possibilidade de exclusão ou alteração de recursos pertencentes a terceiros através de manipulação de parâmetros de identificação de rota.

Correção: Checagem de posse validando providerId e clientEmail no JWT contra o proprietário do registro no PostgreSQL.

[SEC-05] MÉDIA — **Ataques de Força Bruta e Esgotamento de Recursos (DDoS / Spam)**

Rota(s): POST /auth/login, POST /auth/register, POST /auth/social-login, POST /bookings

Falha: Ausência de limitação de tentativas de autenticação e agendamentos abusivos por IP.

Correção: Acoplamento de middleware Sliding Window Rate Limiter com limites granulares e injeção do cabeçalho Retry-After no status HTTP 429.

[SEC-06] MÉDIA — **Prevenção de Conflitos de Horário e Double Booking Concorrente**

Rota(s): GET /slots, POST /bookings, PATCH /bookings/:id/reschedule

Falha: Risco de agendamentos sobrepostos ou descumprimento de tempos de intervalo (buffer pré e pós atendimento).

Correção: Algoritmo que calcula intersecção temporal, folgas cadastradas, buffers dinâmicos e bloqueia conflitos com retorno HTTP 409 Conflict.

3. Bateria de Testes Automatizados e Homologação

' BATERIA DE TESTES DE QA (scratch/test-security-suite.js): 15/15 APROVADOS (100%)

- Teste 1: Rate Limiter bloqueia rajadas de requisições excedentes retornando status HTTP 429 com Retry-After.
- Teste 2: Validação estrita de e-mails rejeitando entradas nulas, vazias ou malformatadas.
- Teste 3: Assinatura e verificação de integridade de tokens JWT rejeitando adulterações de chave secreta.
- Teste 4: Algoritmo anti-conflito bloqueando sobreposição direta e respeitando buffers de segurança.
- Teste 5: Criptografia de senhas com Bcrypt validando hashes seguros e rejeitando senhas incorretas.

4. Compilação, Build e Conclusão

- Verificação de Tipos TypeScript (npm run lint): 0 erros.
- Compilação de Produção (npm run build): Bundles de frontend (Vite) e backend (esbuild) gerados com sucesso.
- Parecer Técnico: A API atende integralmente aos padrões de segurança defensiva, isolamento multi-tenant e robustez transacional para ambiente produtivo.